The 'vibe coding spectrum' approach to AI-assisted software development

https://www.ncsc.gov.uk/blogs/the-vibe-coding-spectrum-approach-to-ai-assisted-software-development

18 June 2026 - Toby W, Principal Security Architect, NCSC

Different code deserves different levels of oversight, so calibrate your approach to ‘vibe coding’ accordingly.



Westend61 via Getty Images

AI-assisted software development has taken off. By now, most people are familiar with the term ‘vibe coding’; giving an AI agent a high-level prompt and letting it build your application with significant autonomy. You prompt, it codes, you review the output, rinse and repeat.

The NCSC have previously written about the effects this could have on cyber security. And if you care about security, vibe coding can feel uncomfortable. AI models are trained on vast amounts of existing code, and some of that has security issues. When you let an AI loose on your code base with minimal oversight, there's a real risk it produces code with security vulnerabilities, with a measurable security gap in AI generated code (pdf).

The other issue is that your code base could become complicated and confusing to understand, leading to further issues down the road with maintainability.

Does this all mean that you shouldn’t use vibe coding? Not necessarily. It depends on what you're building.

Vibe coding isn't always bad

Here's the thing: not all code is security critical.

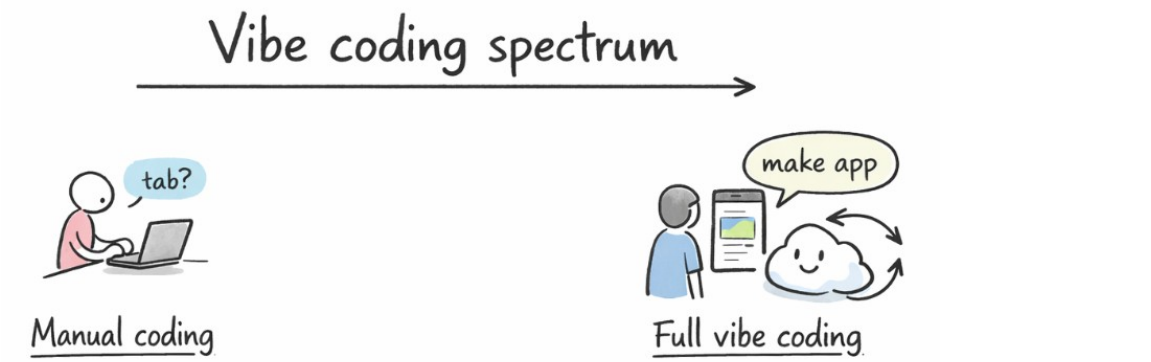
A proof-of-concept to demo an idea to stakeholders? The risk profile is low. An internal tool for your team with limited risk? Maybe that’s manageable. Full vibe coding in these contexts can often be perfectly fine.

But authentication logic for a public facing website? Code that processes sensitive customer data? Anything handling secret tokens or credentials? Safety critical code in a CNI system or code responsible for flying an aircraft? That's a different story. The consequences of getting security wrong in these areas can be severe: data breaches, compromised accounts, regulatory violations or worse.

The risk isn't in using AI. The risk is not applying the right safeguards when the stakes are high. It's about recognising that different code deserves different levels of care and oversight.

The vibe coding spectrum

Vibe coding isn't binary. It exists on a spectrum, and you can position yourself anywhere along it depending on your context and risk tolerance.



- **On the left:** Manual human coding with some AI assistance. You're writing the code yourself, but your IDE offers AI-powered autocomplete. You press tab to accept suggestions, or you don't. You're in full control — the AI is just offering helpful tips.
- **On the right:** Full vibe coding. You provide a high-level, sometimes ambiguous prompt, and the AI has autonomy over architecture, code, modules, and tests. You might iterate through several prompting loops, but you're not editing or deeply reviewing the code — you're evaluating the output and prompting again if needed.
- **In the middle:** A large grey area in between with many variations. Maybe you're asking the AI to write specific functions or methods while you handle the architecture. Maybe you're writing tests manually and letting the AI implement code until those tests pass (test-driven development). Maybe you're specifying modules and reviewing what comes back. There are countless combinations here.

It's about oversight, not avoidance

Let's be clear; this isn't about saying ‘don't use AI for security-critical code’. AI can absolutely help you to write authentication logic, data processing pipelines and check your homework. The question is ‘how can you do this safely?'

When you're building a mock-up to pitch an idea, letting the AI crack on with minimal supervision is fine. When you're building the authentication system that protects your customers' accounts, you need more rigour. You need to:

- **review what the AI produces**
- **Understand the code**
- **Check for vulnerabilities**
- **Verify it does what you expect**

When you do these oversight activities, you could even use another AI agent to help you.

You also need to architect the wider system around your code, to minimise the impact of exploitation.

Choosing your position

The NCSC recommend that you think carefully about what you're building, understand the risk, and then slide along the spectrum accordingly.

For example, you might **slide right (towards 'full vibe') when**:

- building prototypes or proof-of-concepts where speed is of the essence
- creating demos to communicate ideas
- working on internal tools with limited exposure
- no sensitive data or security functions are involved

Slide left (towards manual) when:

- building authentication or authorisation logic
- processing sensitive personal data
- handling secrets, tokens, or credentials.
- anything safety critical, for example in Critical National Infrastructure (CNI).
- the consequences of a security flaw would be significant

The NCSC worked with other governments, industry leaders and cyber security experts in ETSI's 'Technical Committee on Securing AI' to develop Baseline Cyber Security Requirements for AI Models and Systems. You should consider these requirements (pdf) when you think your risk profile moves to the right of this spectrum.

Looking ahead

AI models are improving rapidly. It's possible that we'll trust vibe coding more over time as models become more reliable and their outputs become more trustworthy. The spectrum might shift; what feels risky today might feel routine in a year.

But we're not there yet. Calibrate your approach based on today's reality, not tomorrow's potential.

Toby W, Principal Security Architect, NCSC